

COMPLETE FEATURE REFERENCE

7ALP's — Every Feature, Every Flow

A single reference listing every feature across the customer storefront, admin console, and B2B wholesale portal — what each one does, exactly how a user moves through it step by step, and which backend endpoints power it.

Generated 2026-10-06 · Companion to [7alps-platform-documentation.pdf](#)
(architecture-first reference) · This document is flow-first: organized by what a person does, not by code layer.

Contents

1. How To Read This Document
2. Customer Storefront Features (13)
3. Admin Console Features (13)
4. B2B Wholesale Portal Features (6)
5. Cross-Cutting Systems (Auth, Email, Payments)
6. Content & Marketing Pages
7. Full Route Map (Appendix)

1. How To Read This Document

Every feature below is tagged with who can use it:

PUBLIC anyone, no login **CUSTOMER** requires customer login **ADMIN** requires admin login **B2B** requires B2B member login

Multi-step features include a numbered **Flow** — the literal sequence of clicks/screens a person goes through — followed by the API calls that happen behind the scenes. Endpoints are relative to `/api/v1` unless shown in full.

2. Customer Storefront Features

2.1 Account: Email OTP Login & Signup **PUBLIC**

One flow handles both signup and login — there's no separate "create account" page. The system tells new emails from returning ones automatically.

1 Enter email

At `/customer/login`, the customer types their email. A new email also asks for a name at this step.

2 Verify OTP

A 6-digit OTP is emailed via Resend (`POST /customer/send-otp`). The customer enters it at `/customer/verify-otp` (`POST /customer/verify-otp`), which creates the account on first verification.

3 Add mobile (first time only)

If the account has no mobile number on file, verification returns `{needsMobile: true, pendingToken}` instead of a session — the customer is routed to `/customer/add-mobile` to enter an Indian mobile number (`POST /customer/complete-mobile`, validated against `^[6-9]\d{9}$` and checked for uniqueness).

4 Session starts

Only once both email and mobile are confirmed does the server issue the real `customerToken` httpOnly cookie, and the customer lands back in the app.

Why the two-step token. The pending-mobile step uses a separate, 15-minute-lived JWT that cannot access any customer data, place an order, or otherwise act as a login — so a half-finished signup can't be used as a backdoor session.

2.2 Browse & Filter Catalog PUBLIC

At `/products`: filter by category and price range, free-text search by name, and sort (including "Sort by Rating," backed by the admin-set display rating — see 3.5). Filtering/sorting/searching all query the backend rather than filtering a client-side list, so the catalog scales past what's loaded on one page.

2.3 Product Detail Page PUBLIC

At `/products/:id`: image gallery, description, ingredients, variant picker (each variant carries its own price/MRP/stock), quantity stepper, star rating + review count, published customer reviews, and related products. **Add to Cart** and **Buy Now** both live here.

2.4 Wishlist CUSTOMER

1 Tap the heart icon

Available on any product card (catalog, home, related products) and the product detail page. A guest tapping it is redirected to login first.

2 Toggles instantly

Adds/removes the product from `Customer.wishlist` with an optimistic UI update.

3 Review saved items

The dedicated `/customer/wishlist` page lists everything saved, with add-to-cart shortcuts.

2.5 Cart PUBLIC (persists once logged in as CUSTOMER)

A guest can add items to a cart kept in `localStorage`. On login, that guest cart merges automatically into the customer's server-side cart (`Cart` collection, one document per customer). Quantity steppers add/remove lines; dropping a line's quantity below 1 removes it outright. The cart page also shows a real, admin-configured delivery estimate (see 3.2) and a newsletter signup banner (see 2.12).

2.6 Buy Now CUSTOMER

From a product's detail page, **Buy Now** skips the cart entirely: it carries just that one product/variant/quantity via React Router navigation state straight into Checkout. It never reads or writes the persisted cart — so a Buy Now purchase can't accidentally clear or merge with whatever the customer already has sitting in their cart.

2.7 Checkout CUSTOMER

The heart of the storefront transaction — shipping address, delivery-area check, and payment, all on `/checkout` (also used by Buy Now).

1 Shipping address

Pre-filled from the customer's default saved address if one exists. Can be re-entered via the embedded Google Maps picker (search + draggable pin + reverse geocode) or typed manually; name/phone/line1/line2/city/state/pincode are all validated client-side as the customer types or leaves a field.

2 Delivery-area check NEW

As soon as a state resolves, it's checked against the admin's serviceable-states allow-list (fetched once from `GET /settings/public`). If the state isn't served, a red banner appears — "Sorry, we don't currently accept orders from <state>," listing what is served — and the submit button disables. See Section 5.4 for the full mechanics.

3 Apply a coupon (optional) NEW

A coupon field sits above the totals. The customer types a code, or taps one of the **Available offers** chips: every active coupon, with offers the customer can't use right now (below the minimum, or one-time and already used) greyed out and labelled with what's missing. The discount is previewed through `POST /customer/coupons/preview` and shown as a green Discount line in the summary. Removing it, or changing the cart so it no longer qualifies, is handled automatically. See 3.13 and 5.6.

4 Choose payment method

Cash on Delivery (only shown if the admin has it enabled NEW — 3.3) or **Pay Online** (Razorpay) — a simple radio choice; the order total (including a ₹99 shipping fee waived above ₹999) updates live in the order summary.

5 Place the order

COD: `POST /customer/orders` re-validates the address and the delivery-area check server-side, then atomically builds the order, decrements variant stock, and clears the cart in one MongoDB transaction.

Razorpay: see 2.8 below.

6 Confirmation

Redirects to `/customer/orders` with a success flag; the newly placed order appears at the top with status `Confirmed`.

2.8 Payments — Razorpay & COD CUSTOMER

Cash on Delivery is on by default and needs no setup, but the admin can switch it off entirely (3.3) — leaving Pay Online as the only option. Online payment via Razorpay's Standard Checkout is fully live.

1 Order sized server-side

`POST /customer/payments/razorpay-order` computes the charge itself from the customer's live server-side cart (or the specific Buy Now item) — never trusting a client-supplied amount — and validates the shipping address *and* the delivery-area restriction **before** creating the Razorpay order, so a customer outside the serviceable area is turned away before any charge happens. If a coupon was applied, its code is part of this request, so the amount charged already includes the discount.

2 Razorpay checkout modal

Opens in-page with the order amount/ID; the customer completes payment (card, UPI, netbanking, etc.).

3 Verify & create the order

On success, `POST /customer/payments/razorpay-verify` checks Razorpay's HMAC-SHA256 signature, re-checks the delivery-area restriction and the coupon (if one was applied) as final guards, then creates the real order, decrements stock, and clears the cart — identical to the COD path from that point on.

Test card: Mastercard 5267 3181 8797 5449 , any future expiry/CVV, OTP 1234 (any 4–10 digits works in test mode). Avoid the generic Visa 4111 1111 1111 1111 — it triggers an international-card block on this account.

2.9 Orders, Order Again & Delivery Tracking **CUSTOMER**

At `/customer/orders` : full history, searchable by order ID, product, status, or address. Each order shows itemized products with photos, the expected delivery date, and a visual tracking stepper (Confirmed → Processing → Shipped → Delivered). An order placed with a coupon shows a Discount line with the code. Two one-click actions live here:

- **Order Again** — re-adds every line item from a past order back into the cart at today's prices/stock, then the customer reviews and checks out normally.
- **Rate this product** — appears on each item once its order is `Delivered` , opening the review modal directly (skipped automatically once that product has already been reviewed by this customer).

2.10 Profile & Address Book **CUSTOMER**

At `/customer/profile` : editable display name (email is the verified account identity and can't be changed here; mobile is locked once set). A real address book — add/edit/delete/set-default, each entry fillable via the same Google Maps picker used at checkout. Saving your only address (or leaving none marked default) automatically promotes it to default.

2.11 Product Reviews **CUSTOMER**

1 Submit

From a product detail page, or via "Rate this product" on a delivered order line (2.9) — star rating + written comment.

2 Held for moderation

Saved as `Pending` ; not shown publicly yet. One review per customer per product.

3 Admin publishes or flags

An admin moderates at `/admin/reviews` (3.9) — only `Published` reviews appear on the product page.

2.12 Newsletter Signup **PUBLIC**

The "Stay Connected" banner on the Cart page captures an email via `POST /newsletter/subscribe` into a dedicated subscriber list. Idempotent — resubscribing the same address just succeeds again. Capture-only for now: no unsubscribe flow or campaign sending yet.

2.13 Contact / Quote Request Form **PUBLIC**

At `/contact` (and mirrored on the `/b2b-portal` marketing page): name, email, message/quote request. Saved via `POST /contact` into a Contact collection; optionally emails an admin address if `CONTACT_NOTIFY_EMAIL` is configured. Managed by admin at `/admin/contacts` (3.10).

3. Admin Console Features

Everything below requires an `Admin / SuperAdmin` login at `/admin/login` and lives under `/admin/*` .

3.1 Dashboard ADMIN

Live counts — total customers, orders, products, revenue — plus visual charts, and the three settings cards below (3.2, 3.3, 3.4) surfaced right on the dashboard for quick access.

3.2 Delivery Settings ADMIN

A single number: expected delivery window in days. Feeds the delivery estimate shown to customers on the Cart page and the expected-delivery date shown on every order. Backed by the singleton `Settings` document (`expectedDeliveryDays`).

3.3 Cash on Delivery Toggle ADMIN NEW

Turns COD off entirely, leaving Pay Online as the only checkout option — for example while COD fulfillment isn't ready yet in a given rollout phase. Default is on.

1 Flip the switch

The **Cash on Delivery** card on the Dashboard, right next to Delivery Settings — a single "Accept Cash on Delivery" toggle, saved via the same `PATCH /admin/settings` endpoint.

2 Checkout reacts immediately

When off, the COD radio option disappears entirely at `/checkout` ; a customer mid-checkout with COD already selected is silently switched to Pay Online rather than left on a hidden, stale choice.

3 Enforced server-side too

`POST /customer/orders` — the only endpoint that ever creates a COD order — checks the same setting and rejects with `400` if it's off, so the toggle can't be bypassed by calling the API directly.

3.4 Serviceable States ADMIN NEW

Restrict which Indian states/UTs the store accepts orders from — built this round so the business can ship to a limited set of states while scaling up gradually.

1 Open the Dashboard

The **Serviceable States** card sits right below Delivery Settings.

2 Toggle the restriction on

"Restrict orders to selected states" — off by default (orders accepted from anywhere, same as before this feature existed).

3 Check off served states

A checkbox grid of all 28 states + 8 union territories — **checked means "we ship here,"** unchecked blocks that state. Checked rows are tinted green, Check-all/Clear-all shortcuts speed up setup, and a live "Shipping to X of Y — Z blocked" line confirms the current list at a glance. The toggle refuses to turn on with zero states checked (both in the UI and re-validated server-side), so the admin can't accidentally lock out every customer.

4 Save

`PATCH /admin/settings` writes `serviceableStatesEnabled` + `serviceableStates` to the same `Settings` singleton as Delivery Settings. Takes effect immediately — the very next checkout page load picks it up.

See Section 2.7 and 5.4 for exactly how this is enforced against customers.

3.5 Product Management ADMIN

Full CRUD at `/admin/products`: name, description, category, multiple variants per product (each with its own label/price/MRP/stock), multi-image upload direct to Cloudflare R2, and an admin-set display rating (average + review count) shown on product cards and used for "Sort by Rating" until enough genuine reviews accumulate to compute it for real (2.11).

3.6 Category Management ADMIN

Add/edit/activate/delete product categories at `/admin/categories`. Deleting a category still referenced by any product is blocked with a clear explanatory error rather than silently orphaning products.

3.7 Order Management ADMIN

Every order — storefront and B2B — in one server-side paginated, searchable table at `/admin/orders` (search by order ID, customer, product, address, or payment method). Inline status changes walk an order through `Confirmed` → `Processing` → `Shipped` → `Delivered` or `Cancelled`, driving the tracking stepper customers see (2.9).

3.8 Customer Management ADMIN

Every registered customer at `/admin/customers`, with aggregated order count and lifetime spend per customer.

3.9 Review Moderation ADMIN

At `/admin/reviews`: every submitted review, with one-click Publish, Flag, or Delete — the gate between a customer's submission (2.11) and it appearing publicly on a product page.

3.10 Contact Submissions ADMIN

At `/admin/contacts`: every Contact Us / quote-request submission (2.13), with a New → Read → Resolved status workflow and search.

3.11 Support Tickets ADMIN

At `/admin/support-tickets`: every "Contact Support" message a B2B member sends from their dashboard (4.6), server-side paginated and searchable by subject, message, or the submitting member's name/email/business — same New/Read/Resolved workflow as Contact Submissions.

3.12 B2B Team Management & Quote Approval ADMIN

At `/admin/b2b` — the admin side of the wholesale relationship:

- **Team tab** — create marketing-team logins (B2B accounts are admin-created only, no self-signup); each member's bulk order count and earnings, sortable/filterable by month.
- **Quotes tab** — review a pending quote from any B2B member, optionally adjust the quantity or price, then either **Approve** (atomically decrements stock and creates a real order — see 4.4) or **Reject** with a reason. Searchable by member/business/product.

3.13 Coupons

ADMINNEW

Discount codes customers can apply at checkout. Managed at `/admin/coupons` (sidebar: Coupons).

1 Create a coupon

Choose a code (3–20 letters, numbers, dashes or underscores; stored in capitals), an optional description shown to customers, and the discount: **Percentage off** (1–100) or **Flat amount off** in rupees.

2 Set when it's valid

The minimum order amount, in rupees, measured on the items total before shipping. 0 means any order. Below the minimum the coupon is refused, with the amount still needed.

3 Choose the usage rule

Reusable: each customer can use it on every order. **One-time**: each customer can use it once. A cancelled order doesn't count, so cancelling frees the coupon for that customer again.

4 Activate, deactivate, edit, delete

The switch on each card turns a coupon on or off immediately; inactive coupons are hidden from customers and refused at checkout. Editing affects future orders only, since each order keeps the discount it was placed with. Deleting is allowed only for a coupon that has never been used. A used coupon should be deactivated instead, which keeps its usage count.

4. B2B Wholesale Portal Features

Requires a B2B login at `/b2b/login` (accounts created only by an admin — see 3.12). A B2B login represents an internal marketing-team member selling in bulk on behalf of a business buyer, not the buyer themselves.

4.1 B2B Login

B2B

Email + password (own `b2bToken` cookie, fully separate from Customer/Admin sessions). Forgot-password uses the same OTP-via-email pattern as Admin.

4.2 Catalog & Bulk Pricing

B2B

At `/b2b/catalog`: the same live product catalog customers see. The member picks a variant, a bulk quantity (10+ units), and proposes their own discounted unit price, capped at the listed retail price.

4.3 Request Quote

B2B

1 Build a draft

Products picked in Catalog (4.2) collect into an in-progress quote (kept in Redux state).

2 Add buyer details

At `/b2b/request-quote`: the buyer's business name and delivery address (Google Maps picker included).

3 Submit

Creates a `Quote` document, status `Pending`. Past quotes and their status are listed on the same page, searchable.

4.4 Quote-to-Order Approval B2B / ADMIN

The bridge between a B2B quote and a real order:

1 Admin reviews

At `/admin/b2b`'s Quotes tab (3.12), sees the requested items, quantities, and proposed prices.

2 Approve or reject

Approving atomically decrements stock for every line and creates a real `Order` (`source: 'B2B'` , linked to the originating quote and B2B member). Rejecting requires a reason, visible to the B2B member.

3 Order appears both sides

Shows up in the member's `/b2b/orders` and in the admin's unified `/admin/orders` (3.7) immediately.

4.5 B2B Orders B2B

At `/b2b/orders` : every order created from that member's approved quotes, searchable by order ID or product, with the same status filter as the admin view.

4.6 B2B Dashboard & Support B2B

At `/b2b` : quote counts by status, total orders, and total sales at a glance. The "Contact Support" button opens a modal that submits a real ticket to the admin's Support Tickets queue (3.11).

5. Cross-Cutting Systems

5.1 Three Independent Authentication Systems

Role	Cookie	Who creates the account
Customer	<code>customerToken</code>	Self-signup via email OTP (2.1)
Admin	<code>adminToken</code>	Seeded directly (<code>node dev-data/seedAdmin.js</code>)
B2B	<code>b2bToken</code>	Created only by an Admin (3.12)

A login in one role grants no access whatsoever to the other two — each is checked by its own middleware against its own JWT cookie.

5.2 Transactional Email (Resend)

Every email the platform sends — customer login OTP, admin signup OTP, Admin/B2B forgot-password OTP, and the optional contact-form admin notification — renders through one shared branded HTML template (`backend/utils/emailTemplates.js`) using the site's actual brand colors and logo, with a plain-text fallback alongside.

5.3 Payment Security Model

The backend always computes the charge amount itself from server-side data (the customer's live cart, or the specific Buy Now items) — it never trusts an amount sent by the client, which is standard practice against payment tampering. Razorpay's HMAC-SHA256 signature is verified before any order is created or stock decremented.

5.4 Serviceable-States Enforcement Architecture NEW

One shared backend helper — `backend/utils/serviceability.js`'s `assertStateServiceable()` — is called from all three order-placement entry points, so the restriction can never drift out of sync between payment methods:

Entry point	When checked
<code>POST /customer/orders</code> (COD)	Before the order is built
<code>POST /customer/payments/razorpay-order</code>	Before the Razorpay order — and therefore before any charge — is created
<code>POST /customer/payments/razorpay-verify</code>	Re-checked as a guard in case the admin's list changed in the few seconds since order creation

The frontend independently fetches the same allow-list (`GET /settings/public`) to warn the customer before they even attempt to submit — but the server-side check is what's actually authoritative; the API can't be tricked into accepting a blocked order by skipping the UI.

5.5 Cash on Delivery Toggle Architecture NEW

A single `Settings.codEnabled` flag (default `true`) gates COD independently of the state restriction above — the two features don't interact. The frontend hides the COD radio option when it's off (and reassigns an already-selected COD choice to Razorpay), while `orderController.createOrder` — the only endpoint that ever creates a COD order — rejects with `400` if it's off, regardless of what the UI shows. Razorpay's own two-step flow is a separate payment method entirely and is unaffected by this toggle.

5.6 Coupon Pricing Architecture NEW

Coupons are priced in one place, `orderController.priceOrder` , which every payment path calls: COD (`POST /customer/orders`), Razorpay order creation (`POST /customer/payments/razorpay-order`) and Razorpay verification (`POST /customer/payments/razorpay-verify`). The validation rules live in `utils/coupon.js` .

- **Discount:** a percentage coupon takes that share of the items total, rounded to whole rupees. A flat coupon takes its fixed amount. Either is capped at the items total, so a discount can never make an order negative. Shipping is never discounted.
- **Free-shipping threshold:** the ₹999 rule is judged on the items total before the discount, so a coupon never costs the customer free delivery.
- **Re-checked at order time:** the code is validated again inside the order's MongoDB transaction (active, minimum met, one-time not already used by this customer). The checkout preview is advisory; the server is authoritative.
- **Snapshot:** the order stores the coupon's code, type, value and discount amount (`Order.coupon` , `Order.discountAmount`), so later edits or deletes never change a past order. `Coupon.usedCount` increments in the same transaction.
- **Razorpay caveat:** the amount charged is set when the Razorpay order is created. If the coupon is deactivated while the Razorpay window is open, verification can refuse the order after payment. See the Known Limitations in the platform document.

6. Content & Marketing Pages

Public, brand storytelling and navigation aids that round out the storefront. Most are static copy with scroll-reveal animation only; the homepage is the exception, with one genuinely dynamic section.

6.1 Home PUBLIC — /

Assembles eight sections top to bottom: hero, **Categories** (clickable category tabs), **Our Featured Products**, "Why Choose 7ALP's," a quality-assurance banner, a promotional banner, **Testimonials**, and an FAQ accordion.

The one dynamic section. "Our Featured Products" is the only homepage section that calls the backend — it fetches the real product catalog on mount, derives its category tabs from whatever's actually returned, filters client-side per tab, and shows animated skeleton loaders while loading and an empty-state with a "view all" reset. Every other section (hero, testimonials, FAQs, category tiles) is static copy/imagery.

6.2 Our Process PUBLIC — /our-process

The sourcing/production story, built around **Powder Timeline** — a numbered step sequence (Source → Clean → Dry → Grind, etc.), each step with an image, description, and small key/value chips (sourcing method, temperature). Static content, scroll-reveal animation.

6.3 Why 7ALP's PUBLIC — /why-7alps

Brand positioning, anchored by a dark-themed "us vs. them" **Comparison** table (sourcing, grinding, fillers, drying, fineness, testing) rendered with check/cross icons. Static content.

6.4 Global Trade PUBLIC — /global-trade

Export/trade information for international buyers, including a stylized "Where 7ALP Travels" section — an animated origin marker (pulsing pin on Hyderabad) over a background photo with an Origin/Base/Reach metrics strip. Not an interactive map, purely illustrative. Static content.

6.5 404 Not Found PUBLIC — any unmatched route

A themed not-found page — large "404" watermark, decorative image panel, a "Back to Home" link, and a "Go Back" button. Fully static.

Three unused scaffold folders exist in the codebase (`features/process` , `features/partnerships` , `features/whyChooseUs`) — earlier/duplicate versions of the Our Process and Why 7ALP's pages above, superseded by `ourProcess` and `why7ALPs` . None are wired to any route or rendered anywhere in the app; they're dead code, not a live "Partnerships" page (despite what an older internal reference may have implied).

7. Full Route Map (Appendix)

See `7alps-platform-documentation.pdf` , Section 3, for the complete route table with every path, access requirement, and page — kept there as the canonical technical reference so it isn't duplicated (and risk drifting out of sync) across two documents.

7ALP's Feature Reference — generated from the current codebase. Companion to `7alps-platform-documentation.pdf`.